Taller de APIs (RESTful)

ComCom

Departamento de Computación
FCEyN UBA

Primer Cuatrimestre 2026

Introducción a las APIs

¿Qué es una API, y para qué sirve?

¿Cómo se comunican dos programas que no se conocen?

[1] Visual Studio Code, Zed, los de JetBrains, Neovim, emacs, etc.

[2] Claude, Codex, Gemini, etc.

¿Qué es una API, y para qué sirve?

¿Cómo se comunican dos programas que no se conocen?

Las API (Application Programming Interface) son mecanismos que permiten a dos **componentes** de **software comunicarse** entre sí mientras ambos sistemas **cumplan** con un **contrato/especificación**.

[1] Visual Studio Code, Zed, los de JetBrains, Neovim, emacs, etc.

[2] Claude, Codex, Gemini, etc.

¿Qué es una API, y para qué sirve?

¿Cómo se comunican dos programas que no se conocen?

Las API (Application Programming Interface) son mecanismos que permiten a dos **componentes** de **software comunicarse** entre sí mientras ambos sistemas **cumplan** con un **contrato/especificación**.

Ejemplos

- El lector de tarjetas cuando pagás en un local se comunica con la API de la pasarela de pagos.
- Tu IDE[1] favorito habla mediante una API con tu IA/Agente/LLM[2] de confianza.

[1] Visual Studio Code, Zed, los de JetBrains, Neovim, emacs, etc.

[2] Claude, Codex, Gemini, etc.

Tipos de APIs

Existen varios tipos de APIs, algunas son:

- RPC
- WebSocket
- **RESTful**

Siendo estas últimas las más famosas, y las que nos interesarán en el transcurso del taller.

Existen varios tipos de APIs, algunas son:

- RPC
- WebSocket
- **RESTful**

Siendo estas últimas las más famosas, y las que nos interesarán en el transcurso del taller.

¿Qué es una API RESTful?

Las APIs RESTful son un tipo de API que sigue los principios de diseño de REST (Representational State Transfer).

- Utilizan HTTP para realizar operaciones CRUD (Create, Read, Update, Delete) sobre recursos.
- **Un recurso es cualquier entidad que se pueda identificar, como un usuario, un producto, una publicación, etc.**
  - Se puede pensar *vagamente* como «información almacenada identificable».

Métodos/verbos HTTP

Para entender como funcionan las APIs RESTful, es importante conocer los métodos HTTP más comunes:

¿Cómo devuelven la información?

Las APIs RESTful suelen devolver la información en formato JSON, aunque también pueden usar XML u otros formatos. A su vez, utilizan códigos de estado HTTP.

JSON

```
1 {
2   "id": 1,
3   "name": "Juan Pérez",
4   "email": "jperez@dc.uba.ar",
5   ...
6 }
```

HTTP/1.1 200 OK

¿Cómo devuelven la información?

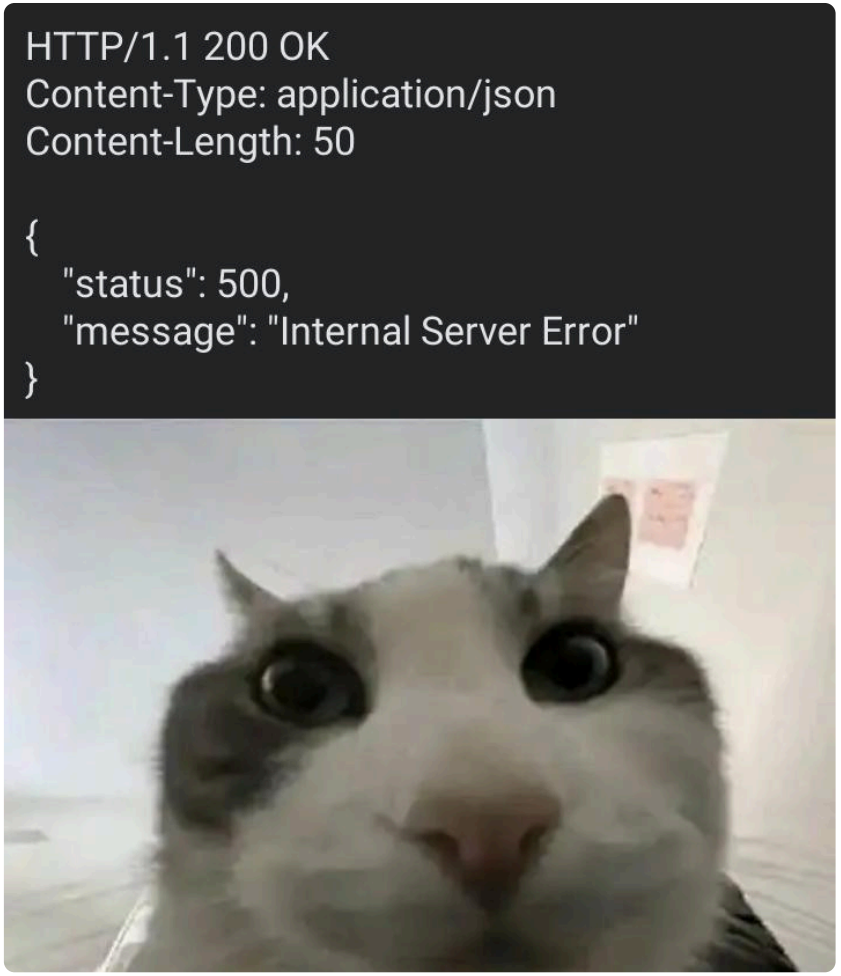
Es importante remarcar que **no toda request devuelve contenido** en el cuerpo.

Por ejemplo, una request de eliminación exitosa del verbo `DELETE` suele devolver un código de estado `204 No Content`, indicando que la operación fue exitosa pero no hay contenido para devolver.

¿Cómo devuelven la información?

Es importante remarcar que **no toda request devuelve contenido** en el cuerpo.

Por ejemplo, una request de eliminación exitosa del verbo `DELETE` suele devolver un código de estado `204 No Content`, indicando que la operación fue exitosa pero no hay contenido para devolver.



También está bueno **ser consistentes** con los códigos de error y **no devolver mensajes redundantes**.

Códigos de estado HTTP

Los códigos de estado HTTP son respuestas estándar que indican el resultado de la solicitud realizada a la API.

- 1xx: Informativos
- 2xx: Éxito
- 3xx: Redirección
- 4xx: Error del cliente
- 5xx: Error del servidor

HTTP status ranges in a nutshell:

1xx: hold on

2xx: here you go

3xx: go away

4xx: you fucked up

5xx: I fucked up

-via @abt_programming

Headers HTTP

Los headers (encabezados) son **metadatos** que viajan junto a la request (cliente → servidor) o la response (servidor → cliente).

Contienen información adicional que no va en el body.

Headers HTTP

Los headers (encabezados) son **metadatos** que viajan junto a la request (cliente → servidor) o la response (servidor → cliente).

Contienen información adicional que no va en el body.

```
GET /me HTTP/1.1
Host: api.ejemplo.com
Authorization: Bearer eyJhbGciOiJIUz...
Content-Type: application/json
```

Headers HTTP

Los headers (encabezados) son **metadatos** que viajan junto a la request (cliente → servidor) o la response (servidor → cliente).

Contienen información adicional que no va en el body.

```
GET /me HTTP/1.1
Host: api.ejemplo.com
Authorization: Bearer eyJhbGciOiJIUz...
Content-Type: application/json
```

Algunos headers comunes:

- `Content-Type`: indica el formato del body (p. ej. `application/json`).
- `Authorization`: información de autenticación y autorización.

Cómo interactuar con una API RESTful

Hay varias formas de interactuar con una API RESTful, algunas de las más comunes son:

- Desde la terminal usando herramientas como `curl` o `httpie`.
- Usando clientes HTTP como Postman o Insomnia.
- Usando la propia documentación de la API, si esta tiene una interfaz interactiva (como Swagger UI o Scalar).

Terminal (p. ej.: curl y httpie)

```
λ ~/ curl http://localhost:3000/items/ \
  --header 'Accept: application/json, multipart/form-data, text/plain'
{"id":"dafe32f6-fcd3-45d3-9fec-ca7ae0fcdee9","value":"ComCom"}%
```

```
λ ~/ http GET http://localhost:3000/items/ \
  Accept:'application/json, multipart/form-data, text/plain'
HTTP/1.1 200 OK
Content-Length: 62
Content-Type: application/json;charset=utf-8
Date: Tue, 21 Apr 2026 17:49:55 GMT

{
    "id": "dafe32f6-fcd3-45d3-9fec-ca7ae0fcdee9",
    "value": "ComCom"
}
```

Clientes HTTP (p. ej.: Postman)

Overview | GET http://localhost:3000/it | No environment

COLLECTIONS
- My Collection

http://localhost:3000/items/ | Save | Share

GET | http://localhost:3000/items/ | Send

Docs | Params | Authorization | Headers 7 | Body | Scripts | Tests | Settings | Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body | Cookies | Headers 3 | Test Results | 200 OK · 72 ms · 184 B

{} JSON | Preview | Visualize

```
1 {
2     "id": "8b952ff6-e6c6-4b62-9f4b-fc53bdb49059",
3     "value": "ComCom"
4 }
```

ENVIRONMENTS
SPECS
FLOWS

Connect Git | Console | Terminal | Globals | Vault | Tools

Documentación interactiva (p. ej.: Scalar)

http://localhost:3000/docs#tag/items/GET/items/

Search ^K Ask AI

Introdu
System
Heal
Items
Get
Crea
Get
Repl
Upd
Dele

GET http://localhost:3000 /items/ Send Open API Client

Get Random Item All

Cookies

Key	Value

Headers

✓	accept	application/json, multipart/form-data, text/plain
	Key	Value

Query Parameters

Key	Value

Code Snippet

9ms 62 B 200 OK All

Cookies
Request Headers 1
Response Headers 3
Body Download

application/json

```
1 {
2   "id": "d9d629c5-0a06-4e1d-aa86-4985d54ef7f5",
3   "value": "ComCom"
4 }
```

Generate MCP

Powered by Scalar

value string required Example

String value to store

```
3   --header 'Content-Type: application/json' \
4   --data '{
```

Interactuando con una API RESTful

Ejercicio 1: Strings anónimos

Esta API RESTful expone:

- `/items`: permite solicitudes `GET` y `POST`.
- `/items/{id}`: permite solicitudes `GET`, `PUT`, `PATCH` y `DELETE`.

Consignas:

- Entren a **/docs** de la API, donde van a encontrar la documentación interactiva.
- Prueben los distintos verbos HTTP sobre las rutas indicadas y observen las respuestas que devuelve la API.[1]

[1] En el taller usaremos la documentación interactiva, pero pueden usar cURL o lo que prefieran.

¿Cómo se estructura un proyecto?

Una forma de organizar el código es separarlo en directorios, cada uno con una responsabilidad:

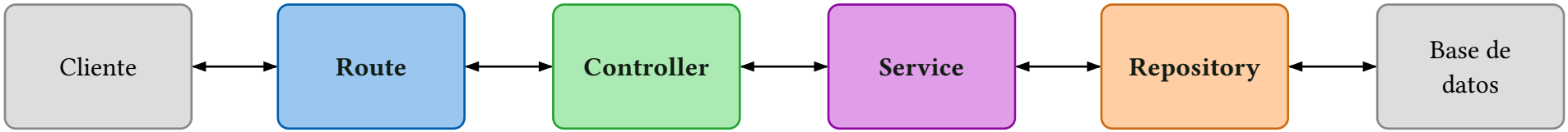
```
1  src/
2  ├── routes/          Definen los endpoints (paths + verbos HTTP)
3  ├── controllers/     Reciben la request y deciden la respuesta
4  ├── services/        Contienen la lógica de negocio
5  ├── repositories/    Acceso a la base de datos
6  ├── schemas/         Definen la forma (y validación) de los datos
7  ├── middlewares/     Funciones que interceptan requests
8  └── databases/       Configuración/conexión a la base de datos
```

También puede estar presente un directorio `utils` para funciones de utilidad.

Generalmente, para cada recurso (p. ej. `items`, `users`, `auth`) existe un archivo en `routes`, `controllers` y `services`; a veces también en `repositories`.

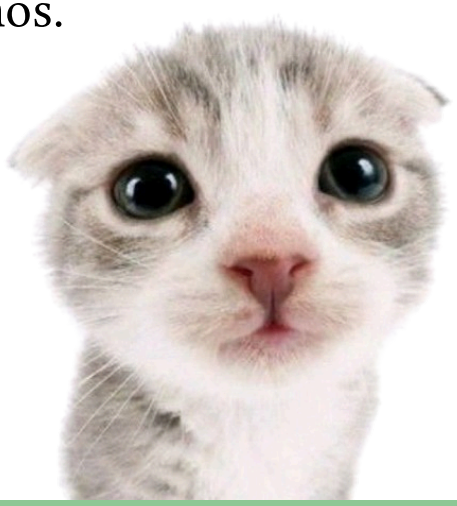
Dependiendo del proyecto, las tecnologías utilizadas o las preferencias del equipo, algunos directorios pueden no existir o pueden tener otros nombres.

Flujo de una request



- La **route** recibe la request y la dirige al **controller**.
- El **controller** decide qué hacer y delega al **service**.
- El **service** ejecuta la lógica de negocio y usa el **repository** para acceder a los datos.
- Los **schemas** validan los datos automáticamente (los vamos a ver más adelante).

Vamos a ir viendo cada una de estas partes a medida que las necesitemos.



Rutas / Routes / Endpoints

Definición

Una ruta (también conocida como endpoint) es una URL que representa un recurso o una colección de recursos en una API RESTful.

- Las rutas se utilizan para acceder a los recursos y realizar operaciones sobre ellos utilizando los verbos HTTP.

Ejemplo

En el ejercicio de los strings anónimos, las rutas expuestas por la API RESTful son:

Ejemplo

En el ejercicio de los strings anónimos, las rutas expuestas por la API RESTful son:

- `/items`: representa la colección de ítems y permite realizar operaciones como obtener un ítem aleatorio de la lista (`GET`) o crear un nuevo ítem (`POST`).

Ejemplo

En el ejercicio de los strings anónimos, las rutas expuestas por la API RESTful son:

- `/items`: representa la colección de ítems y permite realizar operaciones como obtener un ítem aleatorio de la lista (`GET`) o crear un nuevo ítem (`POST`).
- `/items/{id}`: representa un ítem específico identificado por su `id` y permite realizar operaciones como obtener los detalles de un ítem (`GET`), actualizar un ítem (`PUT` o `PATCH`) o eliminar un ítem (`DELETE`).

Route parameters, query parameters, body & response

- Route parameters (p. ej., `{id}`): se utilizan para identificar recursos específicos.
  - `/users/:id`
- Query parameters: se pueden utilizar, por ejemplo, para filtrar, ordenar o paginar recursos.
  - `/users?age=30&sort=asc&page=2`
- Body: se utiliza para enviar datos al servidor en solicitudes `POST`, `PUT` o `PATCH`.
  - `{ "name": "Juan Pérez" }`
- Response: es la información que el servidor devuelve al cliente después de procesar una solicitud.
  - `{ "id": 1, "name": "Juan Pérez" }`
  - `HTTP/1.1 200 OK`

En el template tienen una API con varias rutas ya definidas, por ejemplo `me.routes.ts`:

```typescript
export const meRoutes = new Elysia()
  .use(authenticator)
  .get("/me", meController.obtenerPerfil, obtenerPerfilRouteSchema);
```

Consigna:

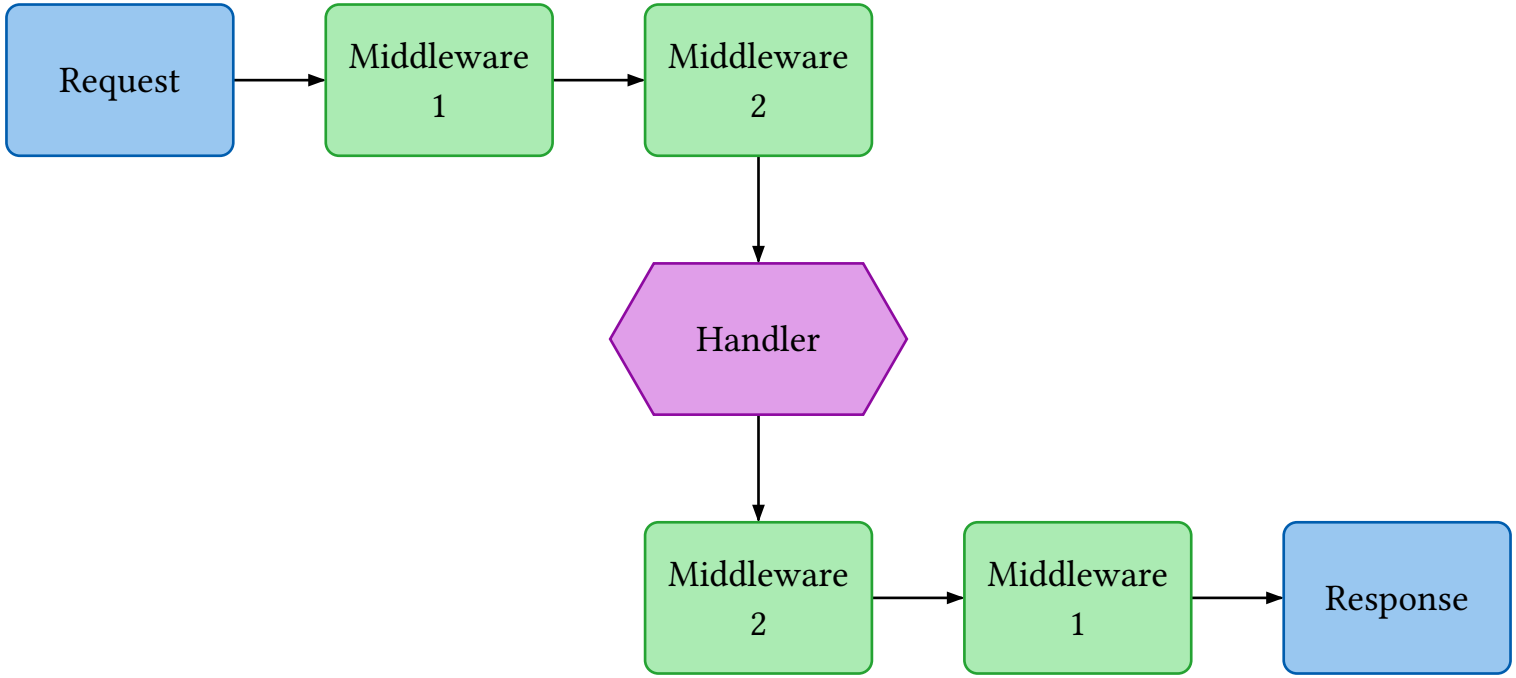
1. Abran `auth.routes.ts`.
2. Registren dos rutas de tipo `POST`:
   - `/auth/register` → `authController.register`
   - `/auth/login` → `authController.login`
3. Por ahora **no** le pasen schemas (tercer argumento). Los vamos a agregar más adelante.
4. Verifiquen que las rutas aparezcan en `/docs`.

Middlewares

¿Qué es un middleware?

Un middleware es una función que se ejecuta **antes** y/o **después** de que un handler procese una request.

Se puede pensar como una «capa» que intercepta las solicitudes y respuestas.



¿Para qué sirven?

Los middlewares permiten agregar funcionalidad a la aplicación sin modificar los handlers individualmente. Por ejemplo:

- **Logging**: registrar cada solicitud que llega al servidor.
- **Autenticación**: verificar que el usuario tenga un token válido.
- **Autorización**: verificar que el usuario tenga permiso para acceder a la ruta.
- **Validación**: verificar que los datos de la solicitud sean correctos.

```typescript
import { Elysia } from 'elysia'

export function middleware_ejemplo(app: Elysia) {
  let middleware_app = app
    .onBeforeHandle(({ request, set }) => {
      // Se ejecuta antes de llegar al handler
    })
    .onAfterHandle(({ request, set }) => {
      // Se ejecuta después de llegar al handler
    });

  return middleware_app;
}
```

Hay más métodos que `.onBeforeHandle` y `.onAfterHandle`, por mencionar uno: `.derive` permite agregar contexto a la request (como por ejemplo, el tiempo exacto en que el cliente mandó la request).

¿Cómo se usa un middleware en Elysia?

Para usar un middleware, simplemente se agrega con `.use()`:

TypeScript

```typescript
import { Elysia } from "elysia";
import { logger } from "./middlewares/logger";
import { authenticator } from "./middlewares/authenticator";

const app = new Elysia()
  .get("/sin-middleware", () => "No uso ningún middleware.")
  .use(logger)
  .get("/con-logger", () => "Uso logger.")
  .use(authenticator)
  .get("/secret", () => "Uso logger y luego authenticator.")
```

Ejercicio 3: Proteger rutas con autenticación

En el template tienen las rutas `/secret` y `/users` **sin protección**...

Consigna:

1. Abran los archivos `secret.routes.ts` y `users.routes.ts`.
2. Importen el middleware `authenticator` de `../middlewares/authenticator`.
3. Agreguen `.use(authenticator)` antes de la definición de la ruta.
4. Prueben acceder a `/secret` **sin** token: deberían recibir un `401 Unauthorized`.
5. Miren cómo se ve `/me` (`me.routes.ts`) como ejemplo de una ruta ya protegida.

Autenticación

¿Cómo sabe la API quién sos?

Las APIs RESTful son **stateless**: no recuerdan quién sos entre request y request.

¿Cómo sabe la API quién sos?

Para resolver esto, se usa un esquema de **tokens**:

1. De una forma u otra, obtenés un **token de acceso** (access token).
2. Luego, en cada request que requiera autenticación, el cliente envía ese token en el header `Authorization`:
   - `Authorization: Bearer <token>`

¿Cómo sabe la API quién sos?

Para resolver esto, se usa un esquema de **tokens**:

1. De una forma u otra, obtenés un **token de acceso** (access token).
2. Luego, en cada request que requiera autenticación, el cliente envía ese token en el header `Authorization`:
   - `Authorization: Bearer <token>`

En nuestra API el **token de acceso** se obtiene mediante el endpoint `POST /auth/login`.

1. El cliente envía sus credenciales (usuario y clave) al endpoint de login.
2. Si son correctas, el servidor devuelve un **token de acceso**.

JWT (JSON Web Token)

El tipo de token más común es el **JWT** (JSON Web Token)[1].

Lo único que interesa para este taller, es que el token es un string que le permite al servidor identificar al usuario, garantizando que no fue alterado.

[1] ¡A pesar de la creencia popular, no es obligatorio usar JWT!

Hashing de claves

Las contraseñas **nunca se guardan en texto plano**. Se guardan como un **hash**: el resultado de una función de una sola vía.

Texto plano (MAL)

```
usuario: valen
clave: 123456
```

Hash (BIEN)

```
usuario: juani
clave: $2b$10$K4f3...
```

Hashing de claves

Las contraseñas **nunca se guardan en texto plano**. Se guardan como un **hash**: el resultado de una función de una sola vía.

Texto plano (MAL)

```
usuario: valen
clave: 123456
```

Hash (BIEN)

```
usuario: juani
clave: $2b$10$K4f3...
```

- Al **registrar**: se hashea la clave antes de guardarla.
- Al **hacer login**: se hashea la clave ingresada y se compara con el hash guardado.
- Si alguien accede a la base de datos, **no puede recuperar las claves originales**.

Controllers, Services y Repositories

¿Qué es un Controller?

Un controller **recibe la request** HTTP, llama al service correspondiente, y **decide qué responder** (código de estado + body).

```typescript
export function obtenerPerfil(contexto) {
  // Llama al service
  return meService.obtenerPerfil(contexto.uuid);
}
```

¿Qué es un Controller?

Un controller **recibe la request** HTTP, llama al service correspondiente, y **decide qué responder** (código de estado + body).

```typescript
export function obtenerPerfil(contexto) {
  // Llama al service
  return meService.obtenerPerfil(contexto.uuid);
}
```

- **No contiene lógica de negocio** (eso va en el service).
- Se encarga de interpretar el resultado y elegir el **status code** adecuado.
- Maneja los **errores** que puede lanzar el service.

¿Qué es un Service?

Un service contiene la **lógica de negocio**: aplica reglas, valida precondiciones y usa el **repository** para acceder a los datos.

```typescript
export function obtenerPerfil(uuid) {
  const user = usersRepository.obtenerPorUuid(uuid);
  if (!user) throw new Error("Usuario no encontrado");
  return user;
}
```

¿Qué es un Service?

Un service contiene la **lógica de negocio**: aplica reglas, valida precondiciones y usa el **repository** para acceder a los datos.

```typescript
export function obtenerPerfil(uuid) {
  const user = usersRepository.obtenerPorUuid(uuid);
  if (!user) throw new Error("Usuario no encontrado");
  return user;
}
```

- **No conoce HTTP**: no sabe de requests, responses, ni status codes.
- **No accede a la base de datos directamente**: usa el repository.
- Lanza **errores** que el controller traduce a respuestas HTTP.

¿Qué es un Repository?

Un repository se encarga del **acceso a los datos**: leer, escribir, actualizar y eliminar registros de la base de datos.

```typescript
export function obtenerPorUuid(uuid) {
  return uuidAUsuario.get(uuid);
}

export function crear(nombreDeUsuario, claveHasheada) {
  const uuid = crypto.randomUUID();
  const usuario = { uuid, nombreDeUsuario, claveHasheada };
  uuidAUsuario.set(uuid, usuario);
  return usuario;
}
```

¿Qué es un Repository?

Un repository se encarga del **acceso a los datos**: leer, escribir, actualizar y eliminar registros de la base de datos.

```typescript
export function obtenerPorUuid(uuid) {
  return uuidAUsuario.get(uuid);
}

export function crear(nombreDeUsuario, claveHasheada) {
  const uuid = crypto.randomUUID();
  const usuario = { uuid, nombreDeUsuario, claveHasheada };
  uuidAUsuario.set(uuid, usuario);
  return usuario;
}
```

- Es el **único** que habla con la base de datos.
- Expone operaciones **CRUD** (Create, Read, Update, Delete).
- No tiene lógica de negocio ni conoce HTTP.

Controller vs Service vs Repository

	Controller	**Service**	**Repository**
¿Conoce HTTP?	✓	×	×
¿Lógica de negocio?	×	✓	×
¿Accede a la DB?	×	×	✓
¿Decide status code?	✓	×	×
¿Valida datos?	Puede[1]	Reglas de negocio	×

[1] Lo ideal es que no tenga que hacerlo o lo haga lo menos posible, ya veremos por qué.

Ejercicio 4: Implementar login y register

Ahora que entienden cómo se organiza, van a implementar la autenticación.

Parte A – Service (`auth.service.ts`):

1. Implementen `registrar`: hasheen la clave (con `await hashearClave(clave)`) y creen el usuario (con `crearUsuario(nombreDeUsuario, claveHasheada)`), luego devuelvan su UUID.
2. Implementen `login`: obtengan el objeto usuario (con `obtenerUsuarioPorNombreDeUsuario(nombreDeUsuario)`) y comparen la clave que les pasaron con la clave guardada (con `await compararClaves(clave, usuario.claveHasheada)`), luego devuelvan su UUID.

Ejercicio 4: Implementar login y register

Ahora que entienden cómo se organiza, van a implementar la autenticación.

Parte A – Service (`auth.service.ts`):

1. Implementen `registrar`: hasheen la clave (con `await hashearClave(clave)`) y creen el usuario (con `crearUsuario(nombreDeUsuario, claveHasheada)`), luego devuelvan su UUID.
2. Implementen `login`: obtengan el objeto usuario (con `obtenerUsuarioPorNombreDeUsuario(nombreDeUsuario)`) y comparen la clave que les pasaron con la clave guardada (con `await compararClaves(clave, usuario.claveHasheada)`), luego devuelvan su UUID.

Parte B – Controller (`auth.controller.ts`):

1. Completen `registrar`: validen que el body exista, luego sus parámetros (que existan, largos mínimos), llamaen al service dentro del try/catch y devuelvan el token con el status code `201` o que el usuario ya existe con el status code `409`, según corresponda.
2. Completen `login`: validen que el body exista, luego sus parámetros (que existan), llamaen al service dentro del try/catch y devuelvan el token con el status code `200` o que las credenciales son inválidas con el status code `401`, según corresponda.

Ejercicio 4: Implementar login y register

Ahora que entienden cómo se organiza, van a implementar la autenticación.

Parte A – Service (`auth.service.ts`):

1. Implementen `registrar`: hasheen la clave (con `await hashearClave(clave)`) y creen el usuario (con `crearUsuario(nombreDeUsuario, claveHasheada)`), luego devuelvan su UUID.
2. Implementen `login`: obtengan el objeto usuario (con `obtenerUsuarioPorNombreDeUsuario(nombreDeUsuario)`) y comparen la clave que les pasaron con la clave guardada (con `await compararClaves(clave, usuario.claveHasheada)`), luego devuelvan su UUID.

Parte B – Controller (`auth.controller.ts`):

1. Completen `registrar`: validen que el body exista, luego sus parámetros (que existan, largos mínimos), llamaen al service dentro del try/catch y devuelvan el token con el status code `201` o que el usuario ya existe con el status code `409`, según corresponda.
2. Completen `login`: validen que el body exista, luego sus parámetros (que existan), llamaen al service dentro del try/catch y devuelvan el token con el status code `200` o que las credenciales son inválidas con el status code `401`, según corresponda.

Prueben: registrar un usuario, hacer login, intentar con datos inválidos.

Ejercicio 4: Implementar login y register

¿Cuántos `if` tuvieron que escribir para validar los datos?

¿Qué pasa si agregan un campo nuevo al body? ¿Y si cambian un largo mínimo?

Schemas

¿Qué es un Schema?

Un schema **define la forma de los datos**: qué campos tiene el body, qué tipos tienen, qué restricciones cumplen, y qué devuelve la respuesta.

TypeScript

```typescript
const bodySchema = t.Object({
  nombreDeUsuario: t.String({ minLength: 3 }),
  clave: t.String({ minLength: 6 }),
});
```

¿Qué es un Schema?

Un schema **define la forma de los datos**: qué campos tiene el body, qué tipos tienen, qué restricciones cumplen, y qué devuelve la respuesta.

```typescript
const bodySchema = t.Object({
  nombreDeUsuario: t.String({ minLength: 3 }),
  clave: t.String({ minLength: 6 }),
});
```

Si la request no cumple con el schema, **el framework la rechaza automáticamente** con un `400 Bad Request`, sin necesidad de escribir un solo `if`.

Sin schema (`register` en `auth.controller.ts`), no entra todo el código en la diapositiva:

```typescript
// ... código anterior
  if (!body) {
    set.status = 400
    return "Falta body en la request"
  }
  const { nombreDeUsuario, clave } = body;
  if (!nombreDeUsuario || !clave) {
    set.status = 400
    return "Faltan el nombre del usuario y la clave en la request";
  }
  if (nombreDeUsuario.length < 3) {
    set.status = 400
    return "El nombre de usuario debe tener al menos 3 caracteres";
  }
// ... una validación similar para la clave y el resto del código
```

Con schema (`register` en `auth.controller.ts`), entra todo el código en la diapositiva:

TypeScript

```typescript
const { body, set } = contexto;
const { nombreDeUsuario, clave } = body;

// Acá estarían los chequeos manuales, pero ya no hacen falta porque el schema se encarga de eso.

try {
  const uuid = await authService.register(nombreDeUsuario, clave)

  const tokenDeAcceso = await crearTokenDeAcceso(uuid)
  set.status = 201
  return {tokenDeAcceso}
} catch {
  set.status = 409
  return "Ya existe un usuario con ese nombre de usuario";
}
```

¿Cómo se usa un Schema?

El schema se pasa como **tercer argumento** en la definición de la ruta:

```typescript
// Antes (sin schema):
.post("/auth/register", authController.register)

// Después (con schema):
.post("/auth/register", authController.register, registrarUnUsuarioSchema)
```

¿Cómo se usa un Schema?

El schema se pasa como **tercer argumento** en la definición de la ruta:

```typescript
// Antes (sin schema):
.post("/auth/register", authController.register)

// Después (con schema):
.post("/auth/register", authController.register, registrarUnUsuarioSchema)
```

Además, el controller puede tipar sus parámetros con el **contrato** del schema:

```typescript
export async function register(
  contexto: Context<RegistrarUnUsuarioRouteContract> // Toma un contexto que cumple con el contrato
): Promise<RegistrarUnUsuarioRouteContract['response'][201 | 409]> // Devuelve 201 o 409, según el contrato
{ ... }
```

Esto da **autocompletado** y **chequeo de tipos** en el editor.

Bonus: documentación automática

Al definir schemas, la documentación interactiva (Swagger / Scalar) muestra **automáticamente**:

- Los campos esperados en el body, con sus tipos y restricciones.
- Las posibles respuestas con sus códigos de estado.
- Ejemplos de uso.

Bonus: documentación automática

Al definir schemas, la documentación interactiva (Swagger / Scalar) muestra **automáticamente**:

- Los campos esperados en el body, con sus tipos y restricciones.
- Las posibles respuestas con sus códigos de estado.
- Ejemplos de uso.

Todo sin escribir documentación a mano. Solo por definir los schemas.

Ejercicio 5 (extra): Escribir los schemas de auth

Parte A – Schemas (`auth.schemas.ts`):

1. **Completen el schema** `const iniciarSesionSchema = { ... }`[1]:
   - Body: `nombreDeUsuario` (string), `clave` (string).
   - Response 200: objeto con `tokenDeAcceso` (string).
2. **Completen el contrato** `interface IniciarSesionRouteContract { ... }`[2]:
   - Body: `nombreDeUsuario` (string), `clave` (string).
   - Response 200: objeto con `tokenDeAcceso` (string).
   - Response 401: string (mensaje de error).

[1] Básense en el schema `const registrarUnUsuarioSchema = { ... }` que ya está implementado.

[2] El contrato `interface RegistrarUnUsuarioRouteContract { ... }` ya está implementado, úsenlo como referencia.

Parte B – Integrar:

1. Modifiquen `auth.routes.ts`: pasen los schemas como tercer argumento a cada `.post()`.
   - `.post("/auth/register", authController.register, registrarUnUsuarioSchema)`
   - `.post("/auth/login", authController.login, iniciarSesionSchema)`
2. Cambien la signatura de las funciones en `auth.controller.ts` para que usen los contratos de los schemas.
   - `export async function register(contexto: Context<RegistrarUnUsuarioRouteContract>): Promise<RegistrarUnUsuarioRouteContract['response'][201 | 409]> { ... }`
   - `export async function login(contexto: Context<IniciarSesionRouteContract>): Promise<IniciarSesionRouteContract['response'][200 | 401]> { ... }`
3. **Eliminen las validaciones manuales** que ahora son redundantes.
4. Vayan a `/docs` y vean la documentación auto-generada.

¿¿¿¿¿¿Preguntas??????

Recursos y bibliografía adicional

- El origen de los principios REST (https://roy.gbiv.com/pubs/dissertation/fielding_dissertation.pdf)

Les dejamos una lista de algunas APIs publicas por si les interesa investigar:

- https://apislist.com/
- https://free-apis.github.io/
- https://api-transporte.buenosaires.gob.ar/
- https://www.mercadopago.com.ar/developers/es/reference
- https://developer.spotify.com/documentation/web-api